Title: The Edge-cloud Continuum Meets Serverless Execution Runtimes: A Framework for Automated Vulnerability Detection

Year: 2021

Authors: K. Silva, L. Banerjee (Institute of Data and Intelligent Systems, Summit College)

Keywords: cloud computing, scheduling, networking, software engineering, operating systems

Abstract:

Automated vulnerability detection remains a critical challenge for the edge-cloud continuum, particularly in e-governance platforms. This paper proposes a framework built on serverless execution runtimes that explicitly accounts for configuration drift detection. We evaluate the framework on the TPC-H benchmark and compare it against 6 baseline methods. Experimental results show that the proposed approach improves 99th-percentile latency by 7.3% while keeping request throughput within acceptable bounds. We further analyse its limitations and discuss implications for practical deployment.

1. Introduction

The rapid maturation of the edge-cloud continuum has changed how researchers and practitioners approach automated vulnerability detection, yet practical deployment is still limited by scalability, cost, and reproducibility concerns. Over the past decade, serverless execution runtimes have attracted sustained attention as a promising route toward solving automated vulnerability detection in realistic settings such as e-governance platforms. Despite considerable progress, existing solutions for automated vulnerability detection tend to degrade when moving from controlled laboratory conditions to production workloads, which motivates a more systematic investigation. A central difficulty is that improvements in 99th-percentile latency frequently come at the expense of resource utilization, forcing designers to accept trade-offs that are poorly understood.

This paper addresses these gaps by studying how serverless execution runtimes interact with log-structured storage engines when applied to automated vulnerability detection. Our work is motivated by the growing demand for e-governance platforms, where reliability and transparency requirements are especially strict. Understanding configuration drift detection is equally important, because it shapes both the theoretical limits and the operational cost of any proposed approach. The remainder of this paper reviews related work, describes the proposed methodology, and concludes with a discussion of limitations and directions for future research.

2. Related Work

Banerjee et al. (2016) introduced an early framework based on serverless execution runtimes, reporting a 41.4% improvement in resource utilization over previous baselines. Subsequent work by Kim et al. (2009) extended this line of research to e-governance platforms, although the evaluation was restricted to small-scale settings. A survey by Okafor et al. (2020) categorised existing approaches to automated vulnerability detection into three families and highlighted the lack of standard benchmarks. Researchers working on the edge-cloud continuum have explored log-structured storage engines as an alternative, with Fernandez et al. (2019) noting that the approach scales poorly beyond 16 components. More recently, Sharma et al. (2020) combined serverless execution runtimes with log-structured storage engines, achieving a 14.7% reduction in overhead on the TPC-H benchmark.

Several studies have criticised the reliance on synthetic workloads, arguing that results on the TPC-H benchmark may not generalise to e-governance platforms. In contrast to these efforts, our work treats automated vulnerability detection and configuration drift detection jointly rather than as independent concerns. Industry reports corroborate these academic findings, pointing to automated vulnerability detection as a persistent obstacle for e-governance platforms. Despite this body of literature, no prior study has systematically compared

serverless execution runtimes and log-structured storage engines under identical experimental conditions.

3. Methodology

We formalise automated vulnerability detection as a constrained optimisation task in which the objective is to maximise 99th-percentile latency while keeping defect detection rate within a prescribed budget. All experiments were conducted on the TPC-H benchmark, which provides 128 labelled instances or configurations covering a broad range of operating conditions. To ensure reproducibility, every experiment was repeated 10 times with independent random seeds, and we report mean values together with 95% confidence intervals. Hyperparameters were selected through grid search over a validation split, and the final configuration was fixed before any evaluation on held-out data.

The core routine applies serverless execution runtimes to reduce the effect of configuration drift detection, followed by a calibration step that stabilises 99th-percentile latency. Our implementation is written in Python and C++, and was executed on a workstation with 64 GB of memory and a single mid-range accelerator. We compare against 6 baseline methods, including a conventional approach built on log-structured storage engines and two recent variants reported in the literature. The proposed design improves 99th-percentile latency by 7.3% relative to the strongest baseline and reduces defect detection rate by 30.4%, a statistically significant difference ($p < 0.05$).

Ablation experiments show that removing the log-structured storage engines component reduces 99th-percentile latency by approximately 7.3%, confirming its contribution. A sensitivity analysis over 500 parameter settings shows that performance remains stable, with variation in resource utilization below 3.6%. Threats to validity include the limited diversity of the TPC-H benchmark and the possibility that deployment in e-governance platforms introduces conditions not captured in our setup.

4. Conclusion

This paper investigated automated vulnerability detection through the lens of serverless execution runtimes, and demonstrated that a carefully designed pipeline can improve 99th-percentile latency by 7.3% on the TPC-H benchmark. The findings suggest that the edge-cloud continuum will continue to benefit from closer integration of serverless execution runtimes and log-structured storage engines, particularly in e-governance platforms. Several limitations remain, including the restricted scale of our evaluation and the simplified treatment of configuration drift detection. Future work will extend the framework to larger deployments, explore tighter theoretical guarantees, and release an open benchmark to facilitate fair comparison. We hope the evidence presented here encourages the community to adopt more rigorous and transparent evaluation practices for automated vulnerability detection.